BDA Assignment 1 — Observations

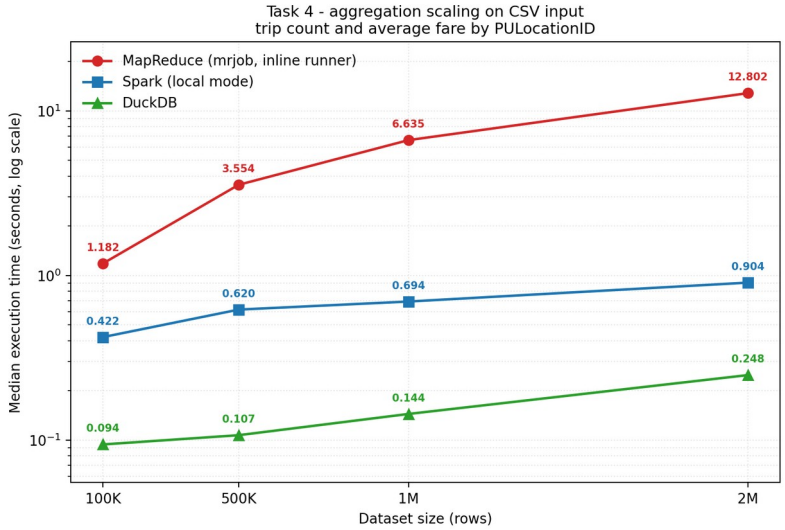
MapReduce vs Apache Spark vs DuckDB | Student ID: 2023481 | AMD Ryzen 7 8845HS, 16 GB RAM, NVMe SSD, Windows 11

1. Predictions made before experimentation

Scenario	Predicted	Reason
Aggregate 1M-row CSV by pickup location	Spark	Spark splits the ~100 MB CSV across partitions and parses it on all 16 logical processors; CSV parsing is CPU-bound and session startup is excluded from the timed region.
Filter 2M-row Parquet, return four columns	DuckDB	Both engines can project columns and skip row groups, so the winner is whichever has less machinery between the query and the file.
Join 2M taxi rows with small zone lookup	DuckDB	The 265-row lookup fits in memory as a hash table; on one machine neither engine must move the large table, so per-query overhead decides.
Scale same aggregation 100K to 2M rows	DuckDB	DuckDB fastest at every size; mrjob steepest because its cost is one interpreted call per row; Spark flattest in relative terms as fixed planning cost dominates at 100K.
Repeat the same query several times	DuckDB	After run 1 the file is in the OS page cache. Spark's cache() removes parsing but must repay a build cost first; DuckDB should stay ahead.

Scenarios 1, 2, 3 and 5 were predicted before any timed run. Scenario 4 was written after Task 1 had been measured at 1M rows, so the ordering at one point was known before the shape was predicted.

2. Task 4 — scaling experiment



Rows	MapReduce	Spark	DuckDB
100,000	1.182	0.422	0.094
500,000	3.554	0.620	0.107
1,000,000	6.635	0.694	0.144
2,000,000	12.802	0.904	0.248

Every curve is close to fixed overhead plus per-row cost. MapReduce grows almost perfectly linearly because its cost is one interpreted Python call, CSV parse and float conversion per row — roughly 6.1 µs, about 75x DuckDB's per-row cost, a ratio that does not improve with scale. Spark's curve is flattest in relative terms, twenty times the data for only 2.1 times the runtime, because at 100K rows about 94% of what was measured is fixed planning and scheduling cost rather than data processing. DuckDB is fastest at every size and is lower on both terms, roughly 0.09 s fixed and 0.08 µs per row, so the fitted lines never cross and more data on this laptop would not let Spark catch up. These local mrjob timings measure the MapReduce programming model executed in Python and must not be read as Hadoop-cluster performance.

3. Median timings for the main comparisons (seconds)

Task / condition	MapReduce	Spark	DuckDB
Task 1 – aggregation, 1M CSV	6.568	0.830	0.148
Task 2 – Query A, all columns	—	0.330	0.078
Task 2 – Query B, four columns	—	0.198	0.025
Task 3 – join + top-5	—	0.678	0.032
Task 5 – repeated query, no cache() (median of 5)	—	0.337	0.004
Task 5 – repeated query, Spark cache() (median of 5)	—	0.278	n/a
Task 5 – Spark cache build, one-off (run 0)	—	1.341	n/a

DuckDB has no user-level cache, so it appears once: 0.004 s is the median of its five repeated runs. Its first run took 0.026 s, six times slower than the rest, which is the OS page cache filling rather than anything DuckDB does.

4. Three observations that surprised me

1. Parquet mattered more than row count. DuckDB filtered 2M Parquet rows in 0.078 s while aggregating 1M CSV rows took 0.148 s — twice the data in half the time, same engine. The 2M Parquet file is also 40.1 MB against 101.7 MB for the 1M CSV.

2. Spark's join was not where the data moved. It broadcast the 12 KB lookup automatically, so the join cost almost nothing; the Exchange in the plan belongs to the groupBy, and it partitions into 200 partitions for a result of a few hundred groups.

3. Spark's cache() only helped by 17%. Median fell from 0.337 s to 0.278 s for a 1.341 s build, needing 22.9 runs to break even. Caching removed re-reading and decoding Parquet — the cheap part — and left planning, scheduling and shuffle untouched.

5. A prediction that was wrong

I predicted Spark for the 1M-row CSV aggregation, reasoning it would parse across all 16 logical processors while session startup stayed outside the timed region. DuckDB won by 5.6x. My error was treating parallelism as Spark's distinguishing feature: DuckDB is also multi-threaded and vectorised and used the same cores, but with no JVM, no per-query Catalyst planning, no task scheduling and no Python-to-JVM boundary. The useful question is not whether a tool can use many cores, but what its design assumes and whether that assumption holds. Spark assumes data too large for one machine; for a 100 MB file that assumption is false, so the architecture built to handle it becomes pure overhead.

6. Projection and filter (Task 2), data movement and join (Task 3)

Projection. Spark's plan shows ReadSchema with 20 columns for Query A and 5 for Query B, and both plans list all four predicates under PushedFilters. The 5 is the point: I asked for four columns, but PULocationID is read as well because the filter needs it, and operator (4) Project drops it afterwards — a query reads what it returns plus what it filters on. Spark keeps a separate Filter above the scan because Parquet pushdown skips whole row groups by min/max statistics, so any group holding one match is read in full and re-checked row by row. DuckDB applies the predicate inside the scan, which is why PULocationID never appears in its Query B projection list at all. In a row-oriented format the two queries would cost the same, since reaching one field means reading past every field in front of it.

Join and data movement. Spark chose BroadcastHashJoin Inner BuildRight without any hint: the lookup is 12,331 bytes against a 10,485,760-byte auto-broadcast threshold, so it ships a 265-row hash table to every task instead of shuffling two million rows. The only Exchange is for the groupBy, and because partial aggregation runs before it, what crosses is one partial sum per group per partition. DuckDB uses the same hash-join strategy with no exchange at all — nothing to exchange between. For MapReduce I would distribute the lookup to the mappers rather than use a reduce-side join: each mapper loads the 265 rows into a dictionary, emits (Borough, Zone) keyed with fare_amount, and reducers sum per key. A reduce-side join would shuffle all two million taxi rows across the network purely to meet a 12 KB table.

7. Task 6 — choosing the tool

	Choice	Justification
A	DuckDB	5 GB fits comfortably beside 16 GB RAM and stays local, and Parquet lets each of the eight queries read only the columns it needs. A cluster would add coordination cost with nothing to coordinate.
B	MapReduce	20 TB already sits distributed across hundreds of machines, so moving it is not an option. A single nightly pass with no iteration is exactly the batch shape MapReduce was built for, and its fault tolerance matters on a long job across many nodes. Spark would also work; MapReduce is sufficient and simpler here.
C	Spark	Repeated joins over large distributed data, feature transformation and model training are multi-stage and iterative, so the same data is read many times. This is where caching genuinely pays: my Task 5 break-even was 23 runs, which iterative training passes far exceeds.
D	DuckDB	No cluster is available, so distribution is not on the table. DuckDB streams a 30 GB CSV without loading it whole and spills to disk if needed. Converting to Parquet first would help further, since five aggregates would then read only the columns involved.
E	DuckDB	2 GB fits on one laptop, so a cluster is unjustified. My own results support this: Spark was 5.6x slower on a 1M-row aggregation, and at 100K rows about 94% of Spark's runtime was fixed overhead rather than data processing.
F	Insufficient information	200 GB alone decides nothing. The answer changes entirely with file format, whether the queries are scans or joins, how much memory is available, and whether the data is already distributed.

8. Conclusion

The factor that mattered most beyond dataset size was whether each tool's design assumption held on this hardware. Spark assumes data too large for one machine; with 100 MB files and 16 GB of RAM that assumption was false, so its planning, scheduling and shuffle became cost without benefit — 94% of its runtime at 100K rows was fixed overhead. Input format came a close second: the same aggregation on the same million rows took 0.148 s from CSV and 0.004 s from cached Parquet, a 37x difference with no change to engine or query. Row count barely moved the ranking; DuckDB led at every size measured. The right question is not which tool is fastest, but what a tool is built to assume and whether that assumption is true of the data, the format and the machine in front of you.